Algorithm 1: High-frequency baseline discrete trading agent (policy gradient)

procedure TRAIN-DISCRETE-POLICY($\{P_t\}$, c, E, K)

    ▷ $\{P_t\}$: `historical price series`

    ▷ c: `transaction-fee coefficient`

    ▷ E: `number of training epochs`

    ▷ K: `number of trading-day episodes per epoch`

    Initialise policy parameters θ and value parameters ϕ randomly
    for *epoch* $e = 1$ **to** E **do**
        for *each trading day (episode)* $k = 1$ **to** K **do**
            Initialise state $s_0 = [\Delta P_0,\ h_0{=}0]^\top$
            Initialise trajectory buffer $\mathcal{B} = \{\}$
            for *minute* $t = 0$ **to** $T - 1$ **do**
                ▷ $T = 390$ `minutes in a US equity session`

                Forward pass: compute $\pi_\theta(a_t \mid s_t)$
                Sample $a_t \sim \pi_\theta(a_t \mid s_t)$
                ▷ $a_t \in \{0\colon \text{Hold},\ 1\colon \text{Buy},\ 2\colon \text{Sell}\}$

                if $a_t = 1$ **then**
                    $h_{t+1} \leftarrow 1$
                    ▷ `Buy / Long`

                else if $a_t = 2$ **then**
                    $h_{t+1} \leftarrow 0$
                    ▷ `Sell / Cash`

                else
                    $h_{t+1} \leftarrow h_t$
                    ▷ `Hold`

                Observe $\Delta P_{t+1} = (P_{t+1} - P_t)/P_t$
                Calculate reward $r_t = (h_t \cdot \Delta P_{t+1}) - c \cdot |h_{t+1} - h_t|$
                Construct $s_{t+1} = [\Delta P_{t+1},\ h_{t+1}]^\top$
                Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{B}$
        Update θ by policy-gradient ascent on $\mathcal{B}$
        ▷ `PPO is one such update rule; treat as black-box optimiser`

        Update value parameters ϕ
    return θ^*

Note for the reader. This is Iteration 1 only: three discrete actions, binary holding $h_t \in \{0, 1\}$, one stock, one day = one trajectory. Later iterations expand the action space; they are not claimed here.